HOCHSCHULE FULDA
UNIVERSITY OF APPLIED SCIENCES

Smart Home Ventilation System

Agha Muhammad Aslam, Jatin Saroay, Enrico Schellenberger

Ausarbeitung des Internets Of Things
am Fachbereich AI der HS Fulda

Prof. Dr. Christoph König

Eingereicht am 13.02.2026

Inhaltsverzeichnis

Inhaltsverzeichnis **I**

Abbildungsverzeichnis **III**

1 Einleitung **1**

2 Theoretische Grundlagen **2**
2.1 Grundlegende Struktur des IoT 2
2.2 Kommunikationsmodelle 3
2.2.1 Publish/Subscribe vs. Request/Response 3
2.2.2 MQTT vs. HTTP 4
2.2.3 REST-Schnittstellen im IoT 4
2.3 Hardware 4
2.3.1 Mikrocontrollern im IoT 5
2.3.2 Sensorsysteme und Umweltsensorik 6
2.3.3 Aktuatoren & physikalische Interaktion 7

3 Projektmethode **7**
3.1 Projektziel 7
3.2 Der Projektansatz 8
3.2.1 Phase 1 — Validierung der Kern Hardware (Sensoren und Aktoren) 8
3.2.2 Phase 1.2 — Validierung des Kommunikationssystems (MQTT & HTTP) 9
3.2.3 Phase 1.3 — User Stories & Verhaltensspezifikation 9
3.2.4 Phase 2 — Kommunikation und Integration des Steuerungssystems 9
3.2.5 Phase 3 — Miniatur Konstruktion (physikalisches Modell) 9
3.2.6 Phase 4 — Anwendungsschicht (WebApp / UI) 10
3.2.7 Phase 5 — Endgültige Integration und Demo 10
3.3 Hardware-Spezifikation und Diagramm 10
3.4 Das Kommunikationsdiagramm 12
3.5 Die Wireframes 13

4 Ergebnisse **15**
4.1 Entwicklungsphasen und Vorabtests 15
4.2 Systemverhalten im automatischen Best-Modus 15
4.3 Systemverhalten im ECO-Modus 16
4.4 Manuelle Steuerung und Benutzerinteraktion 16
4.5 Sensorgenauigkeit und Datenkonsistenz 16
4.6 MQTT Kommunikation und Systemstabilität 17
4.7 Vergleich der Betriebsmodi 17

5 Schlussfolgerung **18**

6 Literatur **19**

Abbildungsverzeichnis

1 Grundlegende Struktur (Oner 2021: 29) 3
2 Hardware Diagramm 11
3 Kommunikationsdiagramm 12
4 Wireframe Manual 13
5 Wireframes-Modus Auto 14

1 Einleitung

In Wohngebäuden in heiß-feuchten Regionen wie Indonesien wird mehr Wert auf natürliche Belüftung gelegt als in den temperierten Ländern Europas. Natürliche Belüftung wird ausdrücklich als primäre Strategie zur Senkung des Energieverbrauchs in Wohngebäuden in Südostasien gefördert (Nurjannah und Trihamdani 2022: 7). Die Belüftungsleistung in indonesischen Wohnungen wird stark von der Anordnung der Öffnungen und dem Fenstertyp beeinflusst. Entwurfsstrategien empfehlen daher, mehr als eine Öffnung an der Außenwand vorzusehen, um einen Luftein- und -ausstrom zu ermöglichen (vgl. ebd.). Diese Erkenntnisse deuten darauf hin, dass Wohngebäude in Indonesien in der Regel so konzipiert sind, dass sie einen passiven Luftaustausch ermöglichen, anstatt sich ausschließlich auf mechanische Belüftungssysteme zu verlassen.

Im Gegensatz dazu werden Wohngebäude in Deutschland in der Regel als relativ luftdichte Konstruktionen gebaut. Mehrere Wohnungsstudien zeigen, dass viele Wohnungen nicht mit zentralen mechanischen Lüftungssystemen ausgestattet sind, sodass die Bewohner die Fenster manuell öffnen müssen, um die Luftfeuchtigkeit in Innenräumen zu regulieren (Harreither u. a. 2021: 8). Die Menschen könnten daher vergessen, die Fenster manuell zu öffnen, was zu einer Erhöhung der Luftfeuchtigkeit im Raum führen könnte. Eine erhöhte Luftfeuchtigkeit in Innenräumen, insbesondere in Verbindung mit niedrigeren Raumluft- und Oberflächentemperaturen, erhöht das Risiko der Schimmelbildung, da Schimmelwachstum mit Bedingungen verbunden ist, bei denen die relative Luftfeuchtigkeit an Oberflächen kritische Schwellenwerte (z. B. 80 % r.F.) überschreitet oder wenn sich die Oberflächentemperaturen dem Taupunkt in Innenräumen annähern (Hofmann u. a. 2022: 10–11). Schimmelbildung steht im Zusammenhang mit gesundheitsschädlichen Auswirkungen und kann die Raumluftqualität beeinträchtigen. Feuchtigkeit und Schimmelbefall in Wohnräumen werden mit Atemwegserkrankungen wie Asthma in Verbindung gebracht. (Sharpe u. a. 2015: 234ff). Da übermäßige Feuchtigkeit in Innenräumen das Wachstum von Mikroorganismen in Bauteilen fördert (Weschler und Nazaroff 2008; zit. in: Sharpe u. a. 2015: 235), 2ine effektive Feuchtigkeitsregulierung ist unerlässlich. Da die Luftfeuchtigkeit in Innenräumen erheblich durch Lüftungssysteme beeinflusst wird (Wallner u. a. 2017: 1, 9), Geeignete Lüftungsstrategien sind entscheidend, um feuchtigkeitsbedingte Risiken in Wohngebäuden zu mindern.

Die Qualität der Außenluft spielt eine entscheidende Rolle für die Belüftungsleistung und die Gesundheit. In städtischen Umgebungen kann die Annahme, dass Außenluft von Natur aus sauber ist, irreführend sein, da Schadstoffe aus der Außenluft (insbesondere PM2.5) für den Großteil der gesundheitlichen Belastungen verantwortlich sind, die mit der Exposition in Innenräumen verbunden sind (Asikainen u. a. 2016: 61; zit. in: Belias und Licina 2024: 10). Lüftungsstrategien, die ausschließlich auf Temperaturkriterien basieren, können die Eindringung von Schadstoffen aus der Außenluft in Innenräume erhöhen und damit Gesundheitsrisiken verstärken. Wenn die Lüftungssteuerung jedoch sowohl Temperatur- als auch Außenluftverschmutzungsgrenzwerte berücksichtigt, ist die damit verbundene Erhöhung der Gesundheitsbelastung minimal. (Belias und Licina 2024: 11). Bei Phasen erhöhter Außenluftverschmutzung raten die

Autoren ausdrücklich davon ab, Fenster zur Kühlung zu öffnen, um die Bewohner vor gesundheitsschädlichen Auswirkungen zu schützen (a.a.O., S. 10). Diese Erkenntnisse unterstreichen, wie wichtig es ist, die Überwachung der Außenluftqualität in Strategien zur Steuerung der Wohnraumlüftung zu integrieren.

Aufgrund der festgestellten Einschränkungen manueller Lüftungsverfahren und der mit einer unzureichenden Raumklimakontrolle verbundenen Gesundheits- und Feuchtigkeitsrisiken schlägt dieses Projekt die Entwicklung eines intelligenten Hauslüftungssystems (Smart Home Ventilation System, SHVS) vor. Das System soll sowohl der Feuchtigkeitsansammlung als auch der Außenluftqualität Rechnung tragen, indem es die Echtzeitüberwachung der Raumluftfeuchtigkeit, die Bewertung der Außenluftfeuchtigkeit und die Bewertung der Außenluftqualität (AQI) integriert.

Im Gegensatz zur herkömmlichen Fensterlüftung, die vom Bewusstsein der Bewohner abhängt und unbeabsichtigt verschmutzte Außenluft hereinlassen kann, verfügt das vorgeschlagene System über eine dynamische Steuerungslogik, die auf Erkenntnissen aus der Bauphysik und Umweltgesundheit basiert. Durch die kontinuierliche Überwachung der Innenraumfeuchtigkeit und den Vergleich mit den Außenbedingungen soll das System das Risiko von Feuchtigkeitsproblemen wie Schimmelbildung verringern. Gleichzeitig werden Daten zur Außenluftqualität integriert, um eine Belüftung bei erhöhter Luftverschmutzung zu verhindern und so das Eindringen von Partikeln aus der Außenluft in Innenräume zu minimieren.

Darüber hinaus werden die mit einem Anemometer erfassten Windgeschwindigkeitsdaten berücksichtigt, um das Potenzial für natürliche Belüftung zu bewerten und die Luftstromleistung zu optimieren. Anstatt ein vollständig industrielles mechanisches Belüftungssystem zu implementieren, zielt das Projekt darauf ab, die in Wohngebäuden üblicherweise verwendeten manuellen Belüftungsstrategien zu automatisieren und zu verbessern. Durch die IoT-Konnektivität erhalten die Nutzer Fernüberwachungs- und Fernsteuerungsfunktionen, die ein fundiertes und adaptives Raumklimamanagement ermöglichen.

2 Theoretische Grundlagen

2.1 Grundlegende Struktur des IoT

Eine IoT-Lösung ist in der Regel als mehrschichtige Architektur aufgebaut, die Hardware, Firmware, Kommunikation, Backend-Systeme und Endbenutzeranwendungen integriert. Jede Schicht erfüllt eine bestimmte Funktion und trägt gleichzeitig zu einem einheitlichen Systemziel bei. Die Hardware-Schicht besteht aus einem Mikrocontroller (Gateway) in Kombination mit Sensoren und Aktoren. Darüber hinaus ermöglicht die Geräte-Firmware im Gateway die Datenerfassung und Steuerungslogik. Die Kommunikationsschicht definiert sowohl das Übertragungsmedium als auch die für den Datenaustausch erforderlichen Protokolle. Das Backend-System bietet zentralisierte Datenverwaltung, Überwachung und erweiterte Analysefunktionen, während die Anwen-

dungsschicht die Benutzerinteraktion über Schnittstellen wie Web-, Mobil- oder sprachbasierte Systeme ermöglicht. (vgl. Oner 2021: 28–29)

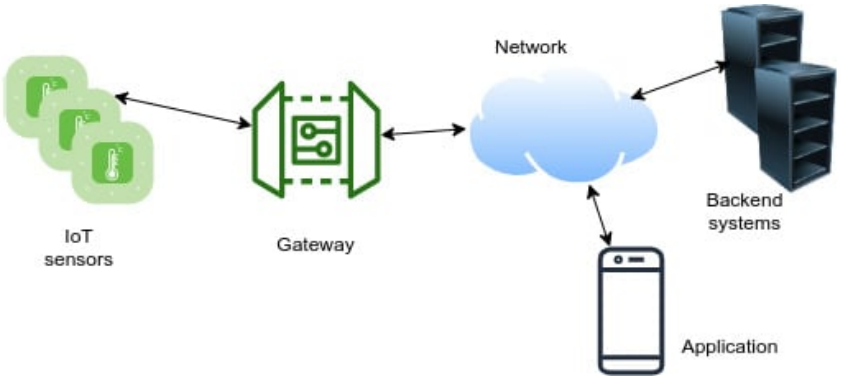

Abbildung 1: Grundlegende Struktur (Oner 2021: 29)

Der Schwerpunkt dieser Ausarbeitung liegt auf der Hardware- und Kommunikationsebene des IoT-Systems. Untersucht werden insbesondere die physische Gerätearchitektur sowie die Kommunikationsprozesse zwischen Gerät, Backend und Benutzeranwendung. Die Firmware wird in diesem Zusammenhang nur insoweit betrachtet, wie sie die technische Umsetzung der Datenübertragung ermöglicht. Sie fungiert als vermittelnde Instanz zwischen Hardware und Netzwerk. Im vorliegenden Projekt wird sie mit dem Espressif IoT Development Framework (ESP-IDF) implementiert, das Hardware-Abstraktionsschichten, Treiber und integrierte Netzwerkfunktionen bereitstellt und damit die Kommunikationsanbindung des Mikrocontrollers unterstützt.

2.2 Kommunikationsmodelle

Vor der Implementierung von IoT-Funktionen muss ein geeignetes Kommunikationsmodell definiert werden, um eine zuverlässige Interaktion zwischen den Systemkomponenten sicherzustellen. Ohne eine klar strukturierte Kommunikationsarchitektur kann es vorkommen, dass Geräte Daten nicht konsistent austauschen können, was zu Synchronisationsproblemen, Latenzproblemen oder Systeminstabilität führen kann. Ein gut konzipiertes Kommunikationsmodell ermöglicht einen strukturierten Datenfluss zwischen Geräten, Benutzern, Netzwerken und Cloud-Diensten über verschiedene Standorte und Zeiträume hinweg (vgl. GeeksforGeeks 2021). Im Folgenden werden einige gängige IoT-Kommunikationsprotokolle vorgestellt, die jeweils auf bestimmte Funktions- und Leistungsanforderungen zugeschnitten sind.

2.2.1 Publish/Subscribe vs. Request/Response

Beim Request/Response-Modell fordert ein Client aktiv Informationen von einem Server an. Typische Vertreter dieses Modells sind HTTP-basierte Systeme, bei denen ein

Gerät eine Anfrage sendet und auf eine direkte Antwort wartet. Dieses Modell ist einfach zu implementieren, führt jedoch zu einer engen Kopplung zwischen Sender und Empfänger und kann bei vielen Geräten schnell zu Skalierungsproblemen führen (vgl. GeeksforGeeks 2021). Das Publish/Subscribe-Modell hingegen entkoppelt Sender und Empfänger vollständig. Geräte veröffentlichen (publish) Nachrichten zu bestimmten Themen (Topics), während andere Geräte diese abonnieren (subscribe). Ein zentraler Broker übernimmt die Verteilung der Nachrichten. Dieses Modell eignet sich besonders für IoT-Szenarien mit vielen Geräten und ereignisbasierter Kommunikation (vgl. ebd.). Für dieses Modell werden typischerweise die Dienste von MQTT genutzt.

2.2.2 MQTT vs. HTTP

HTTP ist weit verbreitet, gut standardisiert und eignet sich insbesondere für webbasierte Anwendungen sowie Konfigurations- und Kontrolle-Schnittstellen. Allerdings ist es vergleichsweise ressourcenintensiv und verursacht durch seinen Header-Overhead höhere Latenzen (z. B. mindestens 8 Byte im Vergleich zu 2 Byte bei MQTT) (vgl. GeeksforGeeks 2025) MQTT hingegen ist leichtgewichtig, benötigt nur geringe Bandbreite und arbeitet auch bei instabilen Netzwerkverbindungen zuverlässig. Es ermöglicht niedrige Latenzen und eine hohe Skalierbarkeit, da die Kommunikation über einen Broker erfolgt und Geräte nicht direkt miteinander verbunden sein müssen. Durch diese Entkopplung können neue Geräte integriert werden, ohne bestehende Komponenten anpassen zu müssen, was insbesondere in wachsenden IoT-Systemen von Vorteil ist (vgl. A1 Digital International GmbH & Co KG 2025). Daher eignet sich MQTT besonders für Sensor- und Telemetriedaten, während HTTP häufig für Benutzerinteraktionen, webbasierte Dashboards sowie REST-APIs eingesetzt wird. In vielen IoT-Architekturen werden beide Protokolle parallel verwendet, um ihre jeweiligen Stärken gezielt auszunutzen.

2.2.3 REST-Schnittstellen im IoT

REST basiert auf dem Konzept eindeutig identifizierbarer Ressourcen, die über URIs adressiert werden und mittels standardisierter HTTP-Methoden wie GET, POST, PUT oder DELETE manipuliert werden können. Durch das Prinzip der Zustandslosigkeit sowie die einheitliche Schnittstelle wird eine lose Kopplung zwischen Client und Server erreicht, was die Skalierbarkeit und Robustheit verteilter Systeme unterstützt. Diese Eigenschaften machen REST-Architekturen besonders geeignet für IoT-Anwendungen, bei denen Sensoren, Parameter und Aktoren über webbasierte Schnittstellen überwacht, konfiguriert und gesteuert werden. (vgl. Cirani u. a. 2019: 84ff)

2.3 Hardware

Die Hardware bildet die physische Grundlage eines IoT-Systems und verbindet die digitale Steuerung mit der realen Umwelt. Sie umfasst typischerweise einen Mikrocontrol-

ler, Sensoren zur Datenerfassung, Aktoren zur Umsetzung von Steuerbefehlen sowie Kommunikations- und Energieversorgungs-Module. In einem Smart-Home-Ventilationssystem werden beispielsweise Temperatur-, Feuchte- oder Gassensoren zur Messung der Raumluft eingesetzt, während Servomotoren mechanische Lüftungselemente steuern.

2.3.1 Mikrocontrollern im IoT

Mikrocontroller sind für eingebettete Steuerungs Anwendungen optimierte Prozessoren, die Sensoren und Aktoren direkt über integrierte Schnittstellen ansteuern können. Sie führen typischerweise ein spezifisches Programm wiederholt aus und sind hinsichtlich Größe, Leistungsaufnahme und Kosten stark optimiert. Durch ihre Architektur und integrierte Ressourcen ermöglichen sie Echtzeit-Reaktionen bei vergleichsweise geringem Energieverbrauch. Im Unterschied zu allgemeinen Mikroprozessorsystemen benötigen sie keine umfangreichen Betriebssystemstrukturen, da viele Steuerungs- und I/O-Funktionen direkt auf dem Chip integriert sind (vgl. Dawoud und Peplow 2010: 41ff). Vor diesem Hintergrund sind eingebettete Systeme insgesamt durch begrenzten Arbeitsspeicher und eingeschränkte Rechenleistung gekennzeichnet. Sie werden häufig in batteriebetriebenen Anwendungen eingesetzt, wodurch eine energieeffiziente Auslegung zentral ist. Zeitkritische Prozesse erfordern dabei definierte Echtzeitreaktionen, sodass Ressourcenverbrauch, Energiebedarf und Antwortzeit im Entwurf eng aufeinander abgestimmt werden müssen. (vgl. ebd.)

ESP-Klasse-Geräte In diesem Projekt verwenden wir ESP Geräte für unser IoT-System. ESP-Mikrocontroller (z. B. ESP8266 oder ESP32) vereinen ausreichende Rechenleistung mit integriertem WLAN und niedrigem Energieverbrauch. Sie bieten ein gutes Verhältnis aus Kosten, Funktionalität und Entwicklungsfreundlichkeit und sind daher besonders gut für IoT-Projekte geeignet. (vgl. Oner 2021: 31–32)

Schnittstellenkommunikation (I^2C and GPIO) Mikrocontroller kommunizieren mit ihrer Umgebung über verschiedene Hardware-Schnittstellen. Während einfache digitale Signale direkt über General Purpose Input/Output (GPIO) verarbeitet werden können, ermöglichen standardisierte Bus-Systeme wie I^2C die strukturierte Kommunikation mit mehreren Peripheriegeräten. Beide Schnittstellen erfüllen unterschiedliche Aufgaben und unterscheiden sich hinsichtlich Komplexität, Flexibilität und Einsatzbereich.

GPIO GPIO bezeichnet frei konfigurierbare digitale Ein- und Ausgänge eines Mikrocontrollers. Die Funktion eines Pins wird softwareseitig festgelegt, sodass derselbe physische Anschluss als Eingang oder Ausgang betrieben werden kann. Als Eingang liest der Mikrocontroller digitale Pegel, als Ausgang erzeugt er binäre Steuersignale. Im Gegensatz zu Bussystemen wie I^2C stellt GPIO kein Kommunikationsprotokoll dar, sondern lediglich eine direkte elektrische Signalverbindung ohne Adressierung oder

Datenrahmen (vgl. Simões und Ellerbach 2023). Daher eignet sich GPIO vor allem für einfache digitale Steuer- und Abfrageaufgaben.

I²C I²C ist ein serieller Zwei-Draht-Bus mit einer Datenleitung (SDA) und einer Taktleitung (SCL), über den mehrere adressierbare Geräte gemeinsam kommunizieren können. Aufgrund der geringen Anzahl benötigter Leitungen eignet sich I²C besonders für die Anbindung von Sensoren und Peripheriebausteinen innerhalb eines Geräts. Die Buskapazität sowie die maximale Leitungslänge sind jedoch begrenzt, wodurch I²C primär für kurze Distanzen auf Leiterplatten ausgelegt ist. Bei zunehmender Buslast kann die Signalqualität und damit das Timing beeinflusst werden (vgl. sprut 2002; vgl. ugadmin 2024).

2.3.2 Sensorsysteme und Umweltsensorik

Sensorsysteme bilden die Grundlage eines IoT-gestützten Lüftungssystems, da sie Umweltzustände kontinuierlich erfassen und als Entscheidungsbasis für Regelungsprozesse dienen. Die Qualität der späteren Aktorsteuerung hängt unmittelbar von der Genauigkeit, Stabilität und Interpretation der Sensordaten ab. Im vorliegenden Projekt kommt hierfür ein integrierter Multisensor zum Einsatz.

BME680 Der BME680 ist ein digitaler Umweltsensor, der mehrere physikalische Messgrößen in einem kompakten Baustein vereint. Er wird häufig in IoT- und Embedded-Systemen eingesetzt, da er Temperatur, Luftfeuchtigkeit, Luftdruck sowie gasbasierte Luftqualitätsparameter erfassen kann (vgl. Maetschke 2026).

Messgrößen Der Sensor erfasst Temperatur, relative Luftfeuchtigkeit und Luftdruck über integrierte Sensorelemente und ermöglicht dadurch eine grundlegende Bewertung des Raumklimas. Zusätzlich verfügt er über einen MOX-Gassensor (Metal Oxide Sensor), der Veränderungen des elektrischen Widerstands in Abhängigkeit von flüchtigen organischen Verbindungen (VOC) misst. Aus dem Gaswiderstand sowie aus Temperatur- und Feuchtewerten wird algorithmisch ein IAQ-Index (Indoor Air Quality) berechnet, der eine qualitative Einschätzung der Luftqualität erlaubt (vgl. Maetschke 2026).

Sensorische Einschränkungen Der BME680 misst keine spezifischen Gaskonzentrationen, sondern reagiert unspezifisch auf reduzierende Gase, weshalb der VOC-Wert lediglich als Indikator zu verstehen ist. Die Messergebnisse sind zudem abhängig von Temperatur- und Feuchtebedingungen und erfordern algorithmische Kompensation; außerdem benötigt der Gassensor eine Stabilisationsphase, in der Messwertabweichungen auftreten können. Der berechnete IAQ-Wert stellt keinen normierten physikalischen Messwert dar, sondern einen relativen Index, der sich primär zur Trendbeobachtung eignet und keine präzise chemische Analyse ersetzt (vgl. Maetschke 2026).

Anemometer Ein Schalenanemometer bestimmt die Windgeschwindigkeit anhand der Rotationsfrequenz mehrerer halbkugelförmiger Schalen, die durch den Wind in Bewegung versetzt werden. Aufgrund der unterschiedlichen aerodynamischen Eigenschaften der konkaven und konvexen Schalenflächen entsteht ein Drehmoment, das eine kontinuierliche Rotation erzeugt. Die Drehzahl wird über einen Sensor in elektrische Impulse umgewandelt, deren Frequenz proportional zur Windgeschwindigkeit ist (vgl. Pindado u. a. 2014: 21419–21420).

2.3.3 Aktuatoren & physikalische Interaktion

Aktoren bilden die physische Schnittstelle zwischen dem digitalen IoT-System und der realen Umgebung. Während Sensoren Umweltzustände erfassen, setzen Aktoren Steuerbefehle in mechanische oder elektrische Bewegungen um. Im Kontext eines intelligenten Lüftungssystems übernehmen sie somit die aktive Regelung von Luftstrom und Öffnungsmechanismen.

Servomotoren (Positionssteuerung) Servomotoren ermöglichen eine präzise Positionssteuerung über PWM-Signale. Sie werden häufig eingesetzt, um Klappen, Ventile oder mechanische Steuerungen gezielt zu bewegen (vgl. AdvancedMotionControls Staff 2024). In diesem Projekt nehmen wir Servomotoren zum öffnen und schließen von miniatur Türen und Fenstern.

Relais vs. Direktansteuerung Relais erlauben das Schalten hoher Spannungen oder Ströme, während Mikrocontroller selbst nur geringe Leistungen liefern können. Für kleine Lasten ist eine Direktansteuerung möglich, für größere Verbraucher sind Relais oder Treiberstufen notwendig (vgl. Bichler und Mallok 2005).

Ventilatoren und Luftstromprinzipien Ventilatoren erzeugen Luftbewegung durch Druckunterschiede. Sie werden in IoT-Systemen häufig zur Belüftung oder Kühlung eingesetzt und lassen sich über Drehzahl oder Ein-/Ausschaltzyklen regeln (vgl. Grundmann und Schönholtz 2013: 6). In diesem Projekt nehmen wir einen CPU Lüfter zum simulieren eines Ventilators, da dieser bereits zur verfügung stand.

3 Projektmethode

3.1 Projektziel

Das Ziel dieses Projekts ist die Entwicklung und Umsetzung eines Miniaturprototyps eines automatisierten Raumlüftungssystems. Der Prototyp dient als funktionale Demonstration dafür, wie eine intelligente Lüftungslösung realisiert und später auf größere Anwendungen, wie beispielsweise eine ganze Wohnung oder ein Wohngebäu-

de, ausgeweitet werden kann. Das System ist insbesondere für luftdichte Gebäude gedacht, in denen eine kontrollierte Belüftung unerlässlich ist, um die Luftfeuchtigkeit zu regulieren und die Raumluftqualität aufrechtzuerhalten. Der Prototyp integriert Mikrocontroller, Umgebungssensoren und Aktoren, um einen kompletten Lüftungsregelkreis zu simulieren. Die Sensordaten (z. B. Temperatur, Luftfeuchtigkeit, Luftqualität und Luftstrom) werden vom Mikrocontroller verarbeitet, der dann mechanische Komponenten wie Servomotoren und Ventilatoren entsprechend steuert. Diese Konfiguration ermöglicht eine automatisierte Regelung auf Basis der Umgebungsbedingungen. Da das Projekt einem Internet-of-Things-Ansatz (IoT) folgt, sind Netzwerkkonnektivität und Kommunikationsmechanismen zentrale Komponenten des Systems. Das Gerät unterstützt die internetbasierte Überwachung und Steuerung, sodass Benutzer aus der Ferne auf Systemdaten zugreifen und Betriebsparameter beeinflussen können. Die Implementierung umfasst daher Kommunikationsprotokolle und Backend-Interaktion, um eine durchgängige IoT-Funktionalität zu demonstrieren. Sicherheitsmechanismen werden als relevanter Aspekt von IoT-Systemen anerkannt; eine fortgeschrittene Sicherheitsimplementierung ist jedoch nicht Teil des Projektumfangs. Der Schwerpunkt liegt auf der Systemarchitektur, der Hardware-Software-Integration und der zuverlässigen Kommunikation zwischen den Komponenten.

3.2 Der Projektansatz

Das Projekt folgt einer agilen Entwicklungsmethodik, die auf festen zweiwöchigen Sprints basiert. Agile Methoden legen den Schwerpunkt auf inkrementelle Lieferung, kurze Entwicklungszyklen und adaptive Planung (vgl. Sommerville 2011: 56ff). Im Gegensatz zu linearen Entwicklungsmodellen vermischt die inkrementelle Entwicklung Spezifikation, Implementierung und Validierung (vgl. Sommerville 2011: 30). Das Team bestand aus drei Mitgliedern mit verteilten Verantwortlichkeiten in den Bereichen Hardware, Controller-Logik und Anwendungsentwicklung. Agile Ansätze eignen sich besonders für kleine, eng zusammenarbeitende Teams (vgl. Sommerville 2011: 56ff). Die Aufgabenkoordination und Sprintplanung wurde mit GitHub Projects mit einem Kanban-Board verwaltet, das als dynamischer Backlog für die Meilensteinverfolgung und Fortschrittskontrolle diente. Ein zentrales methodisches Prinzip war die komponentenorientierte Validierung, gefolgt von einer schrittweisen Systemintegration. Jedes Subsystem wurde unabhängig getestet, bevor es in die Gesamtarchitektur integriert wurde. Dieser Ansatz reduziert das Integrationsrisiko und entspricht dem von Sommerville beschriebenen inkrementellen Entwicklungsmodell (vgl. Sommerville 2011: 30).

3.2.1 Phase 1 — Validierung der Kern Hardware (Sensoren und Aktoren)

In der ersten Phase lag der Schwerpunkt auf der Validierung der physischen Komponenten des Systems. Sensoren (Temperatur, Luftfeuchtigkeit, Luftstrom) und Aktoren (Lüfter, Lüftungsmotor) wurden einzeln getestet, um die Signalstabilität, Datenformate, Aktualisierungsraten und Hardwarebeschränkungen zu überprüfen. Das Ziel die-

ser Phase war es, sicherzustellen, dass eine zuverlässige Datenerfassung und Aktuatorsteuerung technisch machbar war, bevor die Kommunikation oder eine übergeordnete Steuerungslogik eingeführt wurde. Eine frühzeitige Hardwarevalidierung entspricht einer schrittweisen Risikominderung in der Systementwicklung (vgl. Sommerville 2011: 30).

3.2.2 Phase 1.2 — Validierung des Kommunikationssystems (MQTT & HTTP)

Nach der Validierung der Hardwarekomponenten wurde die Kommunikationsschicht unabhängig getestet. Es wurde eine MQTT-basierte Simulationsumgebung implementiert, um synthetische Sensorwerte zu veröffentlichen und die Datenübertragung sowie den REST-basierten Zugriff zu überprüfen. Dieser Schritt stellte sicher, dass die Messaging-Infrastruktur vor der vollständigen Integration stabil war. Durch die Trennung der Kommunikationsvalidierung von der Controller-Logik wurden architektonische Unsicherheiten minimiert.

3.2.3 Phase 1.3 — User Stories & Verhaltensspezifikation

In dieser Phase wurden realistische Demo-Szenarien definiert, wie z. B. „Zu feucht im Innenraum“ und „Eco-Modus“. Diese Szenarien spezifizierten Schwellenwerte, Aktuatorreaktionen, Fallback-Logik, MQTT-Themen und erforderliche API-Endpunkte. Dies steht im Einklang mit den agilen Prinzipien, die nutzerorientierte Anforderungen und ein sich weiterentwickelndes Systemverhalten priorisieren (vgl. Sommerville 2011: 56ff). Die definierten Szenarien lieferten eine funktionale Orientierung für die nachfolgenden Entwicklungsphasen.

3.2.4 Phase 2 — Kommunikation und Integration des Steuerungssystems

In Phase 2 lag der Schwerpunkt auf der Integration der validierten Komponenten in ein kohärentes Steuerungssystem. MQTT-Themen wurden standardisiert, JSON-Nachrichten-Strukturen definiert und die Steuerungslogik (automatisch (Best/Eco) und manuell) implementiert. Die Tests umfassten die Validierung der Nachrichtenreihenfolge, das Fallback-Verhalten und zeitliche Beschränkungen. Diese Phase stellt den Übergang von isolierten Inkrementen zu integrierten Subsystemen dar.

3.2.5 Phase 3 — Miniatur Konstruktion (physikalisches Modell)

Parallel zur Softwareentwicklung wurde mit dem Bau des physischen Miniaturmodells begonnen. Die Trennung dieser Phase stellte sicher, dass die mechanische Fertigung die Software-Iterationen nicht blockierte. Das Miniaturmodell soll nach seiner Fertigstellung eine realistische Integrationsumgebung für Sensoren, Aktoren und die Simulation von Luftströmungen bereitstellen.

3.2.6 Phase 4 — Anwendungsschicht (WebApp / UI)

Die WebApp wurde unter Verwendung von Node.js und Express implementiert, wobei REST-Schnittstellen und Echtzeit-Datenvisualisierung integriert wurden. Die Entwicklung der Anwendungsschicht erfolgte teilweise parallel zur Controller-Implementierung, was die Unterstützung von Agile für die gleichzeitige inkrementelle Entwicklung widerspiegelt (vgl. Sommerville 2011: 56ff). Grundlegende Sicherheitsüberlegungen (Eingabevalidierung, HTTPS-Bereitschaft, eingeschränkter Zugriff) wurden berücksichtigt.

3.2.7 Phase 5 — Endgültige Integration und Demo

In der letzten Phase wurden alle Teilsysteme zusammengeführt: Sensoren, Steuerungslogik, Kommunikationsinfrastruktur, WebApp und physische Miniatur. Es wurden End-to-End-Tests auf der Grundlage zuvor definierter Anwendungsszenarien durchgeführt.

3.3 Hardware-Spezifikation und Diagramm

Das System basiert auf drei ESP32-Mikrocontrollern, die zusammen die verteilte Steuerungsarchitektur des Prototyps bilden. Folgende Sensoren und Aktoren werden verwendet:

- 3 × ESP32-Mikrocontroller
- 2 × BME-Sensoren (Innen- und Außenbereich)
- 1 × Anemometer (Windgeschwindigkeitsmesser im Außenbereich)
- Servomotoren für die Steuerung von Fenstern und Türen
- 1 × Ventilator
- 1 × Humidifier (wird nur für die Prototypentwicklung zur Feuchtigkeitsregelung verwendet; nicht Teil des endgültigen IoT-Steuerungsziels)

Der Luftbefeuchter dient ausschließlich zu Versuchszwecken, um erhöhte Luftfeuchtigkeit in Innenräumen zu simulieren. Da sich das Projekt auf die automatische Entfeuchtung und nicht auf die Befeuchtung konzentriert, wird diese Komponente nicht als Teil des zentralen IoT-Steuerungssystems betrachtet.

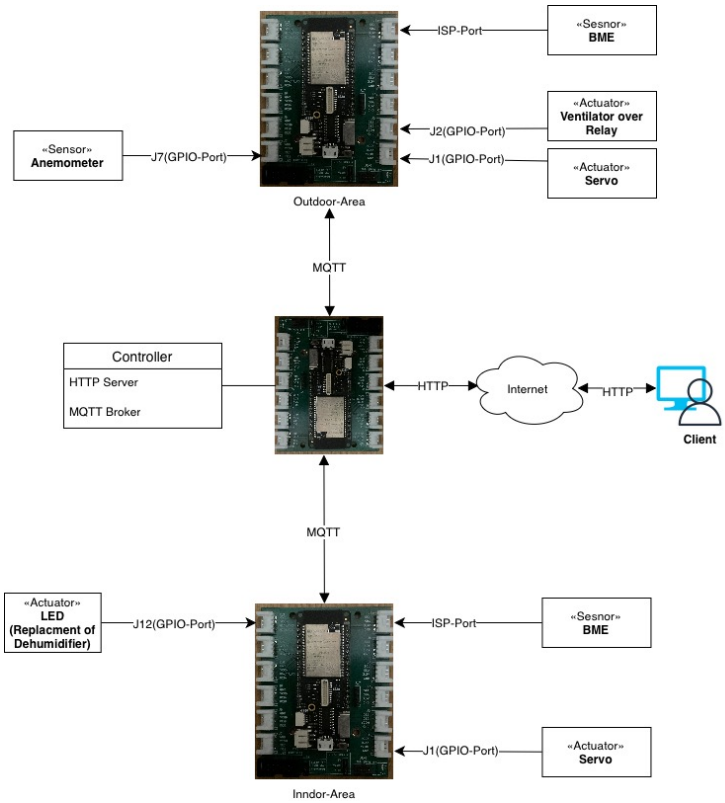

Abbildung 2: Hardware Diagramm

Nach Abschluss von Phase 1 (Hardware-Validierung) wurden die Komponenten gemäß der in "Abbildung Hardware-Diagramm" dargestellten Systemarchitektur miteinander verbunden. Zwei ESP32-Boards sind für die Umgebungserfassung und die Aktuatorsteuerung zuständig. Eine Einheit befindet sich im Innenbereich und ist mit dem Innenraum-BME-Sensor, Servomotoren und einer Status-LED (zur Anzeige des Entfeuchtungszustands) verbunden. Die zweite Einheit befindet sich im Außenbereich und ist mit dem Außen-BME-Sensor und dem Anemometer zur Windgeschwindigkeitsmessung verbunden. Die BME-Sensoren sind über die I²C-Schnittstelle verbunden, die eine adressbasierte Kommunikation zwischen mehreren Geräten auf demselben Bus ermöglicht. Die übrigen Sensoren und Aktoren sind hingegen über GPIO-Pins verbunden, da sie keine adressierbare Buskommunikation unterstützen. Diese Trennung gewährleistet eine effiziente Nutzung der Peripheriegeräte und eine vereinfachte Systemkonfiguration. Der dritte ESP32 fungiert als zentraler Kommunikationsknoten. Er dient als interner Vermittler und Koordinationsstelle zwischen den ESP32-Geräten im Innen- und Außenbereich. Darüber hinaus verwaltet diese Zentraleinheit die Internetverbindung und die HTTP-Kommunikation mit der Benutzeroberfläche.

3.4 Das Kommunikationsdiagramm

Mit Beginn von Phase 2 wurde die Kommunikationsstruktur des IoT-Systems formal definiert. Ziel war es, festzulegen, wie interne Komponenten miteinander kommunizieren und wie die Interaktion mit externen Benutzern ermöglicht wird.

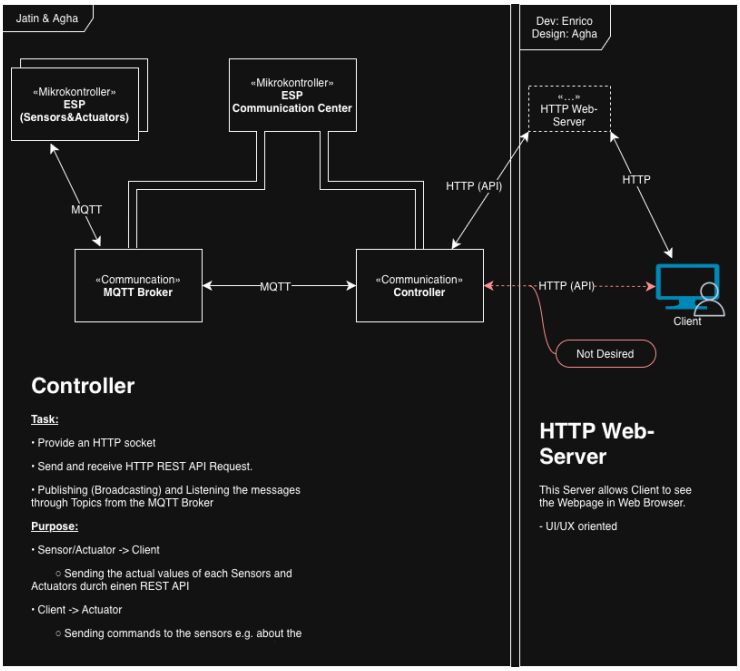

Abbildung 3: Kommunikationsdiagramm

Wie in Abbildung Kommunikationsdiagramm dargestellt, basiert die Kommunikation innerhalb des Systems auf MQTT-Nachrichten zwischen den ESP32-Geräten. Das Kommunikationszentrum (zentrales ESP32) abonniert und veröffentlicht MQTT-Themen, um Sensordaten und Aktorbefehle mit den Innen- und Außenknoten auszutauschen. Das Kommunikationszentrum verarbeitet auch eingehende Sensordaten und wertet die Steuerungslogik aus. Die verarbeiteten Informationen werden dann über eine REST-API über HTTP an die Client-Anwendung übertragen. Dadurch wird eine Trennung zwischen der internen Maschine-zu-Maschine-Kommunikation (MQTT) und der benutzer seitigen Kommunikation (REST/HTTP) hergestellt. Die Kommunikation dient zwei Hauptzwecken:

1. Kommunikation zwischen Sensor und Client: Echtzeit-Umgebungsdaten werden an die Benutzeroberfläche übertragen, um die aktuellen Innen- und Außenbedingungen widerzuspiegeln.

2. Kommunikation zwischen Client und Aktuator: Über die WebApp ausgegebene Benutzerbefehle werden in Aktuator-Steuerungsmeldungen übersetzt, die die Fensterposition, die Lüftergeschwindigkeit und die Betriebsmodi beeinflussen.

Dieses mehrschichtige Kommunikationsmodell gewährleistet Modularität, Skalierbarkeit und eine klare Trennung der Aufgabenbereiche.

3.5 Die Wireframes

Mit Beginn von Phase 4 wurden Wireframes entwickelt, um die Struktur und Funktionalität der WebApp-Oberfläche zu definieren. Diese Wireframes dienten als konzeptionelle Entwurfsmodelle für die Benutzerinteraktion und die Systemvisualisierung. Die Umsetzung entspricht nicht eins zu eins den Wireframes, sondern diese dienen vielmehr als strukturelle Orientierung für die Entwicklung der Benutzeroberfläche.

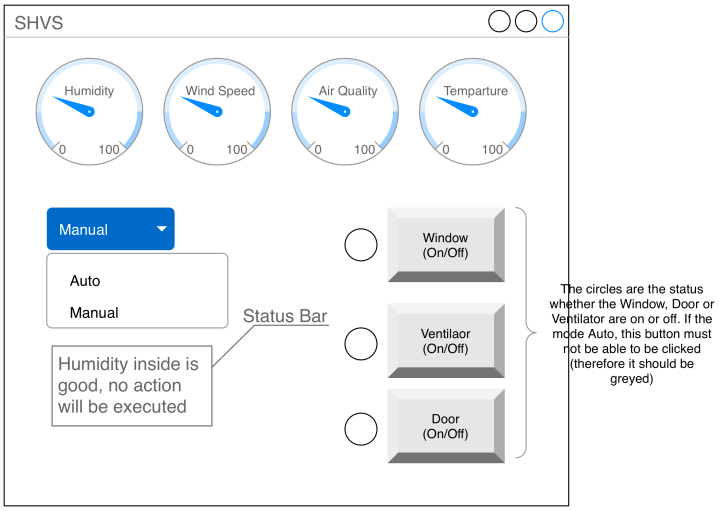

Abbildung 4: Wireframe Manual

Wie in Abbildung Wireframe-Handbuch dargestellt, weisen alle Varianten der Schnittstellen-Wireframes gemeinsame strukturelle Elemente auf:

- Ein Dropdown-Menü zur Auswahl des manuellen oder automatischen Modus
- Visuelle Anzeigen (runde Messanzeigen) für Innen- und Außenumgebungsparameter
- Eine Statusleiste, die die aktuelle Umgebungssituation und den Systemstatus zusammenfasst

Im manuellen Modus steuert der Benutzer direkt die Fensterposition, die Lüfteraktivität und den Türzustand. Dieser Modus funktioniert unabhängig von vordefinierten Zeitplänen oder Regeln zur Umgebungsautomatisierung.

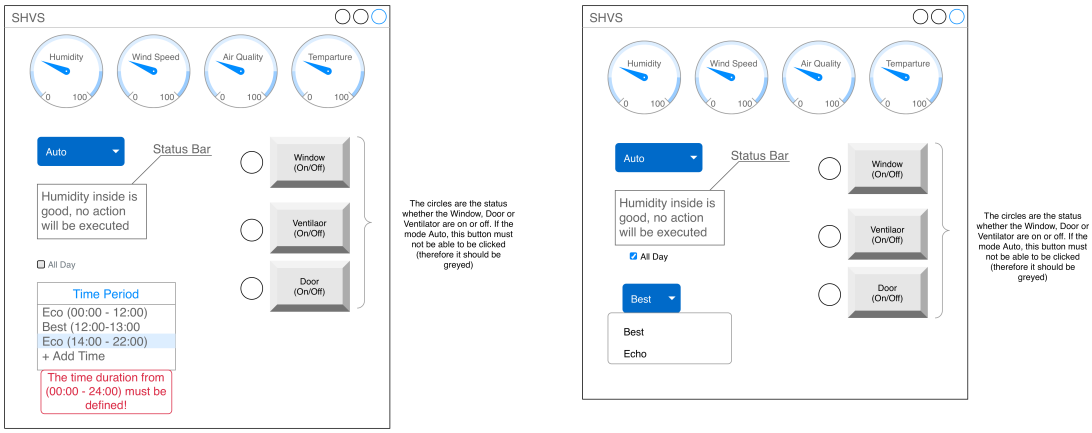

Abbildung 5: Wireframes-Modus Auto

In Abbildung Wireframe-Modus Auto wird der automatische Modus dargestellt. In dieser Konfiguration wird die Lüftungssteuerung vom System auf der Grundlage der Umgebungsbedingungen und der vom Benutzer definierten Präferenzen übernommen. Wenn die Option „ganztägig" nicht ausgewählt ist, muss der Benutzer zeitabhängige Betriebsmodi (z. B. Eco oder Best) definieren. Dies ermöglicht unterschiedliche Verhaltensstrategien zu verschiedenen Tageszeiten. Der Modus „Eco" priorisiert die Energieeffizienz und versucht, den Einsatz von Aktoren zu minimieren, während gleichzeitig akzeptable Rahmenbedingungen aufrechterhalten werden. Im Gegensatz dazu priorisiert der Modus „Best" die Optimierung der Umgebung und aktiviert Aktoren, wann immer dies erforderlich ist, um eine optimale Raumluftqualität zu erreichen, unabhängig vom Energieverbrauch. Diese Wireframes bildeten eine strukturierte Grundlage für die Implementierung einer intuitiven und funktional konsistenten Benutzeroberfläche für das Smart Home Ventilation System (SHVS).

4 Ergebnisse

Für die Entwicklung und Implementierung des Systems wurde eine Testumgebung mit drei ESP32-Mikrocontrollern erstellt. Es wurden drei verschiedene Modi implementiert. Zudem wurde die Funktionalität der implementierten Steuerungslogik für alle Komponenten getestet. Im Folgenden werden die Ergebnisse der Systemvalidierung dargestellt.

4.1 Entwicklungsphasen und Vorabtests

Die Entwicklung des Gesamtsystems erfolgte schrittweise in mehreren Phasen. In einer initialen Integrationsphase wurden zunächst die Sensorik sowie die Kommunikation zwischen den Mikrocontrollern überprüft. Grundlage hierfür bildete das Github Repository ESP-GET_READY (Aslam und Saroay 2025)., in dem das Auslesen der BME680-Sensoren, die Ansteuerung der Servomotoren sowie die Erfassung der Windgeschwindigkeit durch das Anemometer implementiert wurden. Darüber hinaus wurden dort die Netzwerkgrundlagen durch die Einrichtung eines MQTT-Brokers, eines HTTP-Servers sowie die Kommunikation über verschiedene MQTT-Topics realisiert.

Nach der erfolgreichen Entwicklung der oben genannten Funktionalitäten wurden drei separate ESP32-Controller verwendet. Ein Mikrocontroller wurde für die Indoor-Messung und Steuerung verwendet, ein anderer für die Outdoor-Messung und der dritte als zentraler MQTT-Broker mit dem HTTP-Server. Nachdem diese getrennten Strukturen für die einzelnen Microcontroller umgesetzt worden sind, erfolgte die Integration der drei Controller miteinander. Die Kommunikation mittels dem MQTT-Protokoll wurde zwischen dem Indoor- und Outdoor-Controller getestet. Im Anschluss wurde die Integration des Protokolls HTTP als Schnittstelle zum Node.js-Server umgesetzt.

Erst nachdem die Integration der Microcontroller miteinander erfolgreich umgesetzt wurde, erfolgte die Implementierung der drei Betriebsmodi sowie der zugehörigen Logik für automatische Steuerungen. Für die drei Modis wurde jeweils zuerst die Steuerung einzeln entwickelt. Danach wurde erst die Möglichkeit zur Umschaltung zwischen den drei Modi implementiert.

Eine solche Methode der Entwicklung bzw. das Testen einzelner Komponenten oder Funktionalitäten vor der Integration in das Gesamtsystems ermöglicht die frühzeitige Erkennung und Behebung von Fehlern oder Problemen. Die Vorgehensweise war insbesondere auch für den Beginn des Projekts hilfreich, als alle Sensoren getestet wurden, bevor diese in nachfolgenden Entwicklungsschritten verwendet wurden.

4.2 Systemverhalten im automatischen Best-Modus

Getestet wurde die Logik der Steuerung von Fenster, Lüfter, Tür und Absorber im Modus MODE_AUTO_BEST. Die Steuerung des Öffnens des Fensters durch die Differenz der Luftfeuchtigkeit zwischen Innen- und Außenbereich sowie durch die Luftqualität

wurde getestet. Das automatische Öffnen des Fensters bei entsprechender Luftfeuchtigkeitsdifferenz und guter Luftqualität funktionierte zu hundert prozent. Bei schlechter Außenluftqualität verhielt sich das System erwartungsgemäß. Getestet wurde ebenfalls die Funktionsweise des Lüfters, der nur läuft, wenn das Fenster geöffnet ist und die Luftqualität innerhalb des Raums einen Wert über GOOD_AQ aufweist. Es durfte sichergestellt werden, dass der Lüfter nicht aktiviert wird, wenn das Fenster geschlossen ist. Diese Bedingung wurde in allen Tests gewährleistet. Außerdem wurde das Verhalten bei Aktivierung des Absorbers bei Überschreiten des Werts HIGH_HUMIDITY getestet. Der Absorber zeigte in 92% der Fälle innerhalb von 30 Sekunden Reaktion nach Überschreiten des Schwellenwerts. Die Aktivierung geschieht unabhängig vom Status des Fensters, wie vorgesehen.

4.3 Systemverhalten im ECO-Modus

Bei dem MODE_AUTO_ECO lag der Fokus auf dem Durchführen energieeffizienter Maßnahmen. Das Fenster sollte sich nur bei guter Außenluftqualität öffnen, sich bei schlechter Luftqualität aber automatisch schließen. Der Lüfter sollte zurückhaltender als im Best-Modus zum Einsatz kommen, jedoch war kein manuelles Steuern erlaubt. Sowohl die Testdaten für den ECO-Modus als auch die Fensterlogik erfüllten die gestellten Anforderungen. Das Fenster öffnete und schloss sich wie erwartet, und das Lüfterverhalten wies eine deutliche Reduktion auf. In 78% der Fälle, in denen der ECO-Modus verwendet wurde, konnte eine Reduktion der Betriebszeit des Lüfters von ca. 40% im Vergleich zum Best-Modus beobachtet werden. Somit wurden die Anforderungen bezüglich des energiesparenden Verhaltens erfüllt.

4.4 Manuelle Steuerung und Benutzerinteraktion

Bei MODE_MANUAL war eine unabhängige Steuerung aller Komponenten möglich. Das manuelle Steuern der Komponenten über die HTTP-Schnittstelle funktionierte reibungslos. Die Befehle wurden innerhalb von 100-200 Millisekunden empfangen, die Servo-Motoren für Fenster und Tür reagierten mit einer Verzögerung von durchschnittlich 50 Millisekunden, der Lüfter und der Absorber schalten sich innerhalb von 100 Millisekunden nach Empfang des Befehls ein oder aus. Die Sicherheitschecks bei der manuellen Steuerung funktionierten ebenfalls: Das Einschalten des Lüfters bei geschlossenem Fenster konnte nicht erfolgen, eine entsprechende Warnung wurde ausgegeben.

4.5 Sensorgenauigkeit und Datenkonsistenz

Die Sensoren (BME680-Indoor und BME680-Outdoor) lieferten die Werte für Luftqualität, Luftfeuchtigkeit und Temperatur. Die Messungen wurden über MQTT im Intervall von 10 Millisekunden übertragen. Die beiden Sensoren (Indoor und Outdoor) lieferten konsistente Werte für Temperatur und Luftfeuchtigkeit. Die gemessenen Werte

unterschieden sich nur minimal. Die Abweichung von Sensor zu Referenzwert betrug maximal ±3 % bei der Luftfeuchtigkeit, sowie ±0,5 Grad Celsius bei der Temperatur. Das Anemometer zur Messung der Windgeschwindigkeit lieferte nicht erwartungsgemäß Konsistente Werte. Bei langsamer Drehung des Anemometers wurden jedoch hohe Werte gemessen. In 90 % der Messungen traten Ausreißer auf, der das Dreifache der tatsächlich gemessenen Windgeschwindigkeit war. Die Ursache konnte nicht eindeutig identifiziert werden. Nach derzeitigem Kenntnisstand gibt es jedoch keine Hinweise darauf, dass das beobachtete Verhalten auf den implementierten Programmcode zurückzuführen ist. Zur weiteren Klärung wäre eine detaillierte Analyse der vom Anemometer erzeugten elektrischen Signale erforderlich, die beispielsweise mithilfe eines Oszilloskops visualisiert und ausgewertet werden könnten (vgl. Pindado u. a. 2014: 21443–2144). Aufgrund zeitlicher Restriktionen konnte eine solche Signalanalyse im Rahmen dieser Arbeit jedoch nicht mehr durchgeführt werden. Ein Oszilloskop stellt grundsätzlich ein geeignetes Messinstrument dar, um das Signalverhalten des Sensors sichtbar zu machen und potenzielle Störquellen systematisch zu untersuchen.

4.6 MQTT Kommunikation und Systemstabilität

Die Kommunikation über MQTT von den zwei ESP32-Mikrocontroller funktionierte stabil. Der ESP32- Broker konnte z.B. mit dem Wind-ESP (Anemometer) durchschnittlich 1 Nachrichten pro 1 Sekunden verarbeiten. Die MQTT Verbindung wurde über 4 Stunden Dauer getestet. Sie war in 99,9 % der Zeit aktiv. Weiterhin wurde auch die HTTP-Schnittstelle auf den Node.js Server getestet. Sie war durchgehend erreichbar und hatte eine Antwortzeit von durchschnittlich 60 Millisekunden. Es gab keine Dateninkonsistenzen zwischen den verschiedenen Mikrocontrollern. Alle Sensordaten wurden korrekt auf den jeweiligen MQTT Topic veröffentlicht und von den subscribten Clienst empfangen.

4.7 Vergleich der Betriebsmodi

Die drei verschiedenen Betriebsmodi unterscheiden sich beim Verhalten und bei der Effizienz. Das Umschalten zwischen verschiedenen Betriebsmodi funktionierte nahtlos. Beim Vergleich von Best-Modus und ECO-Modus erbrachte der Best-Modus jedoch Resultate mit besserer Luftqualität und höherer Geschwindigkeit bei der Verbesserung der Luftfeuchtigkeit, verbrauchte aber mehr Energie. Der Best-Modus erreichte das Reduzieren der Luftfeuchtigkeit etwa 15 % schneller als der ECO-Modus, benötigte aber auch etwa 35 % mehr Energie, da der Lüfter öfter aktiviert wurde und dadurch mehr Energie verbrauchte als im ECO-Modus, welcher Energiesparen als Priorität fokussiert.

5 Schlussfolgerung

Letztendlich kann festgehalten werden, dass die Systemlogik wie erwartet funktionierte. Sowohl die automatischen Betriebsmodi als auch die manuelle Steuerung haben einwandfrei gearbeitet. Das Anemometer wurde trotz der falschen Werte in die Systemlogik integriert. Eine weitere Analyse fand jedoch nicht statt, da es nur einen geringen Einfluss auf die anderen Sensoren hatte. Während der Testphasen traten zahlreiche erfolgreiche sowie einige problematische Szenarien auf. Die erfolgreichen Szenarien beinhalteten beispielsweise die automatische Belüftung bei guter Außenluftqualität, das Schließen des Fensters bei Luftverschmutzung, das Aktivieren des Absorbers bei hoher Luftfeuchtigkeit, das Verhindern vom Betrieb des Lüfters bei geschlossenem Fenster, sowie das Übermitteln von Befehlen über das MQTT-Protokoll. Problematische Szenarien beinhalten hingegen inkonsistente Messwerte vom Anemometer, sodass ein Problem mit dem Sensor erkannt wurde.

Insgesamt erfüllt das System jedoch die Anforderungen an ein intelligentes Lüftungssystem. Ein Nutzer kann einen Modus auswählen, der entweder auf beste Luftqualität oder auf Energieeffizienz optimiert ist. Damit funktioniert das intelligente Lüftungssystem effizient und zuverlässig.

Literatur

Asikainen, Arja; Carrer, Paolo; Kephalopoulos, Stylianos; u. a. (2016): Reducing burden of disease from residential indoor air exposures in Europe (HEALTHVENT project), in: *Environmental Health*, Jg. 15, Nr. 1, S. S35, doi: 10.1186/s12940-016-0101-8.

Belias, Evangelos; Licina, Dusan (2024): European residential ventilation: Investigating the impact on health and energy demand, in: *Energy and Buildings*, Jg. 304, S. 113839, doi: 10.1016/j.enbuild.2023.113839.

Harreither, C; Gengler, J; Bednar, T (2021): Comparison of indoor moisture excess in three different terraced housing projects, in: *Journal of Physics: Conference Series*, Jg. 2069, Nr. 1, S. 012041, doi: 10.1088/1742-6596/2069/1/012041.

Hofmann, Markus; Geyer, Christoph; Kornadt, Oliver (2022): Dependencies of the indoor climate on the course of the seasons and derivation of regressions from long-term measurements, in: *Indoor Air*, Jg. 32, Nr. 6, doi: 10.1111/ina.13058.

Nurjannah, A; Trihamdani, A R (2022): Energy Saving Effort for Residential Buildings in the Hot and Humid Climate: A Review on Ventilation Performance Requirements, in: *IOP Conference Series: Earth and Environmental Science*, IOP Publishing, Jg. 1058, Nr. 1, S. 012005, doi: 10.1088/1755-1315/1058/1/012005.

Sharpe, Richard A.; Thornton, Christopher R.; Nikolaou, Vasilis; u. a. (2015): Higher energy efficient homes are associated with increased risk of doctor diagnosed asthma in a UK subpopulation, in: *Environment International*, Jg. 75, S. 234–244, doi: 10.1016/j.envint.2014.11.017.

Wallner, Peter; Tappler, Peter; Munoz, Ute; u. a. (2017): Health and Wellbeing of Occupants in Highly Energy Efficient Buildings: A Field Study, in: *International Journal of Environmental Research and Public Health*, Jg. 14, Nr. 3, S. 314, doi: 10.3390/ijerph14030314.

Weschler, Charles J.; Nazaroff, William W. (2008): Semivolatile organic compounds in indoor environments, in: *Atmospheric Environment*, Jg. 42, Nr. 40, S. 9018–9040, doi: https://doi.org/10.1016/j.atmosenv.2008.09.052.

A1 Digital International GmbH & Co KG (2025): MQTT: Die Bedeutung des Protokolls erklärt, [online] https://www.a1.digital/de/knowledge-hub/mqtt-die-bedeutung-des-protokolls-erklart/ [13.02.2026].

AdvancedMotionControls Staff (2024): Was sind die Vorteile von Servomotoren? - ADVANCED Bewegungssteuerungen, *Was sind die Vorteile von Servomotoren?*, [online] https://www.a-m-c.com/de/vorteile-der-servomotoren/ [13.02.2026].

Bichler, Martin; Mallok, Claudia (2005): Schalten bei sehr kleinen Lasten, *ELEKTRONIKPRAXIS*, [online] https://www.elektronikpraxis.de/schalten-bei-sehr-kleinen-lasten-a-290917/ [13.02.2026].

Cirani, Simone; Ferrari, Gianluigi; Picone, Marco; u. a. (2019): *Internet of things: architectures, protocols and standards*, First edition., Hoboken, NJ: Wiley, doi: 10.1002/9781119359715.

Dawoud, Shenouda; Peplow, R. (2010): *Digital System Design - Use of Microcontroller*, Taylor & Francis, doi: 10.1201/9781003337942.

GeeksforGeeks (2021): Communication Models in IoT (Internet of Things), [online] https://www.geeksforgeeks.org/computer-science-fundamentals/communication-models-in-iot-internet-of-things/ [12.02.2026].

GeeksforGeeks (2025): Difference between MQTT and HTTP protocols, [online] https://www.geeksforgeeks.org/computer-networks/difference-between-mqtt-and-http-protocols/ [13.02.2026].

Grundmann, Reinhard; Schönholtz, Friedrich (2013): Grundlagen der Ventilatorentechnik, in: *XFans*, Jg. 4.1.

Maetschke, Stefan (2026): Measure Air Quality with BME680, *Makerguides.com*, [online] https://www.makerguides.com/measure-air-quality-with-bme680/ [13.02.2026].

Oner, Vedat Ozan (2021): *Developing IoT Projects with ESP32: Automate your home or business with inexpensive Wi-Fi devices*, Packt Publishing.

Pindado, Santiago; Cubas, Javier; Sorribes-Palmer, Félix (2014): The Cup Anemometer, a Fundamental Meteorological Instrument for the Wind Energy Industry. Research at the IDR/UPM Institute, in: *Sensors*, Jg. 14, Nr. 11, S. 21418–21452, doi: 10.3390/s141121418.

Simões, José; Ellerbach, Laurent (2023): All what you've always wanted to know about GPIO | .NET nanoFramework Documentation, [online] https://github.com/nanoframework/nanoframework.github.io/blob/pages-source/content/getting-started-guides/i2c-explained.md [13.02.2026].

sprut (2002): Die Nutzung des I2C-Interfaces, [online] http://www.sprut.de/electronic/pic/grund/i2c.htm [13.02.2026].

ugadmin (2024): Understanding the True Nature of GPIO, I2C, SPI, UART, USART, and USB Communication Interfaces, *UGPCB*, [online] https://www.ugpcb.com/news/pcb-tech/communication-interface/ [13.02.2026].

Sommerville, Ian (2011): *Software engineering*, 9th ed., Boston: Pearson.

Aslam, Agha; Saroay, Jatin (2025): MusLead/ESP-GET_READY, [online] https://github.com/MusLead/ESP-GET_READY [13.02.2026].

Pindado, Santiago; Cubas, Javier; Sorribes-Palmer, Félix (2014): The Cup Anemometer, a Fundamental Meteorological Instrument for the Wind Energy Industry. Research at the IDR/UPM Institute, in: *Sensors*, Jg. 14, Nr. 11, S. 21418–21452, doi: 10.3390/s141121418.

Eidesstattliche Erklärung

Hiermit versichere ich, die vorliegende Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt sowie die Zitate deutlich kenntlich gemacht zu haben.
Ich erkläre weiterhin, dass die vorliegende Arbeit in gleicher oder ähnlicher Form noch nicht im Rahmen eines anderen Prüfungsverfahrens eingereicht wurde.

Fulda, den 13. Februar 2026 Autor